

BLOCKCHAIN EXPERIMENT - 7

AIM: Implement the embedding wallet (Metamask) and transaction using Solidity

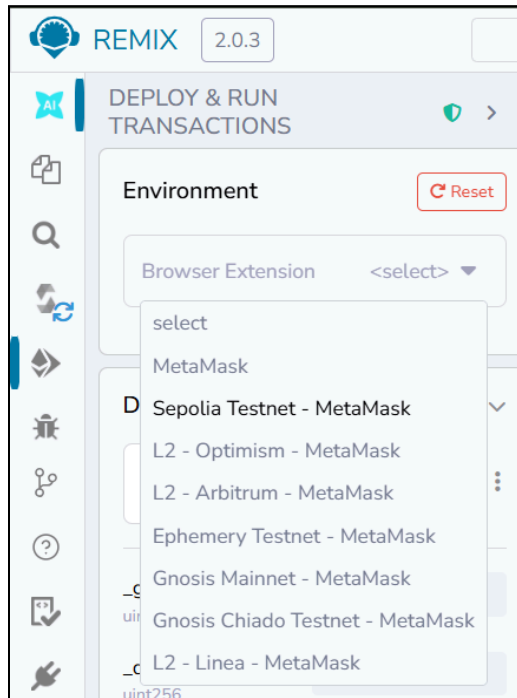
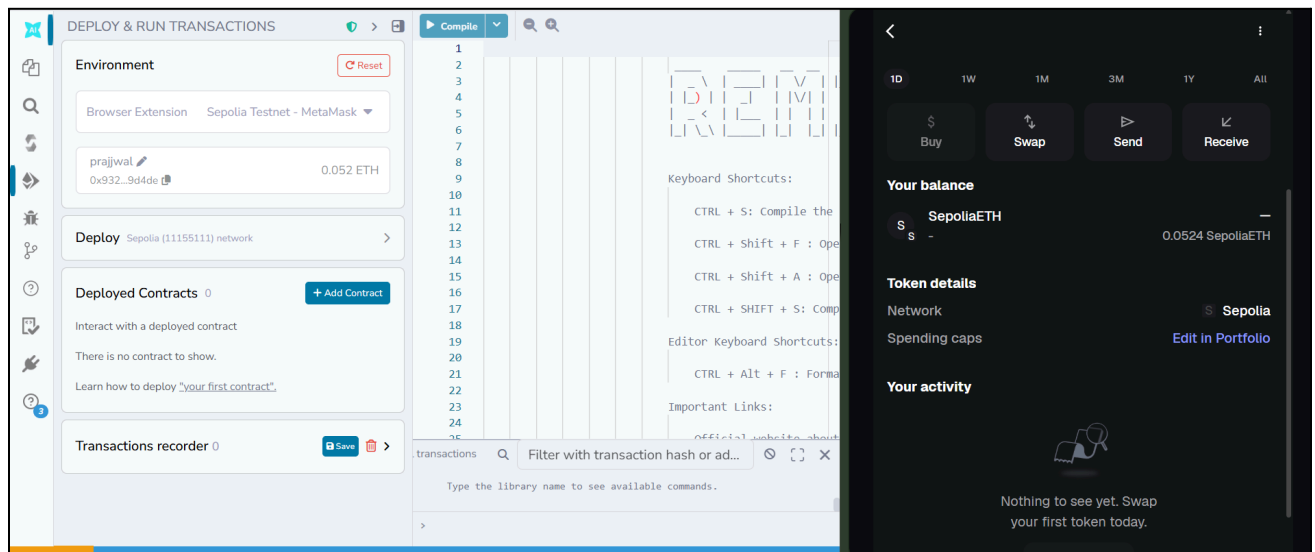
THEORY:

Q1: What is MetaMask? MetaMask is a widely used browser extension that functions as a digital wallet for Ethereum and other compatible blockchain networks. It allows users to manage their accounts, store cryptocurrencies, and send or receive tokens directly from the browser. Beyond basic wallet functionality, MetaMask serves as a gateway between the user and decentralized applications (DApps), enabling seamless signing and authorization of transactions without exposing private keys to third parties. It integrates smoothly with development tools like Remix IDE, making it an essential tool for smart contract deployment and blockchain interaction.

Q2: What is a Test Network (Testnet)? A testnet is a parallel blockchain environment designed specifically for development and experimentation. It replicates the behavior of the Ethereum mainnet but operates using valueless test currency, ensuring that developers can simulate real transactions without any financial consequences. Popular Ethereum testnets include Sepolia and Goerli. These networks allow developers to deploy smart contracts, test their logic, observe gas consumption, and debug errors in a realistic setting before releasing anything on the actual mainnet. Testnets are an essential step in the smart contract development lifecycle, significantly reducing the risk of costly mistakes in production.

Q3: Steps to Connect MetaMask with Remix IDE for Performing Transactions

- Download and install the MetaMask extension from the Chrome Web Store and set up a new wallet by following the onboarding steps
- Securely note the Secret Recovery Phrase provided during wallet creation, as it is required to restore access if needed
- Navigate to MetaMask settings and enable the option to show test networks
- Switch the active network to a testnet such as Sepolia from the network dropdown
- Visit a Sepolia faucet website and request free test ETH by entering your wallet address, which will be used to pay gas fees during testing
- Open Remix IDE in the browser at remix.ethereum.org and create or open a Solidity smart contract file
- Use the Solidity Compiler tab in Remix to compile the contract and resolve any errors
- Navigate to the Deploy & Run Transactions tab and choose Injected Provider - MetaMask from the Environment dropdown
- Approve the connection request that MetaMask displays to link your wallet with Remix
- Select the appropriate contract from the dropdown, click Deploy, and confirm the transaction in the MetaMask popup
- Once deployed, use the contract's functions visible in the Deployed Contracts section and approve each transaction through MetaMask as required

IMPLEMENTATION:**Step 1: Select metamask sepolia testnet in environment****Initial setup:**

Step 2: Create a contract for a crowdfunding platform**CODE:**

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.0;

contract Crowdfunding {
    address public owner;
    uint public goal;
    uint public deadline;
    uint public totalFunds;

    mapping(address => uint) public contributions;

    constructor(uint _goal, uint _duration) {
        owner = msg.sender;
        goal = _goal;
        deadline = block.timestamp + _duration;
    }

    function contribute() public payable {
        require(block.timestamp < deadline, "Deadline passed");
        require(msg.value > 0, "Send some ETH");

        contributions[msg.sender] += msg.value;
        totalFunds += msg.value;
    }

    function withdrawFunds() public {
        require(msg.sender == owner, "Only owner");
        require(totalFunds >= goal, "Goal not reached");

        (bool success, ) = payable(owner).call{value: totalFunds}("");
        require(success, "Transfer failed");
    }

    function refund() public {
        require(block.timestamp > deadline, "Not ended");
        require(totalFunds < goal, "Goal was reached");

        uint amount = contributions[msg.sender];
        require(amount > 0, "No contribution");
    }
}
```

```

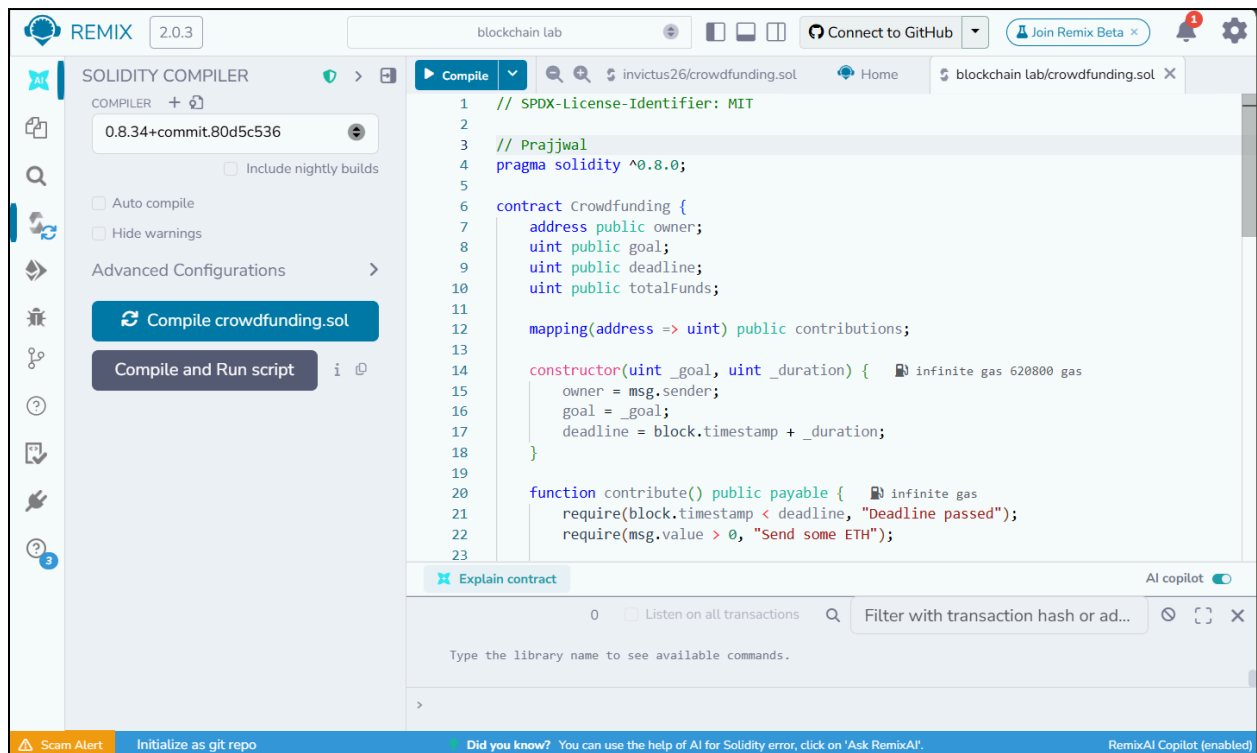
contributions[msg.sender] = 0;

(bool success, ) = payable(msg.sender).call{value: amount}("");
require(success, "Refund failed");
}

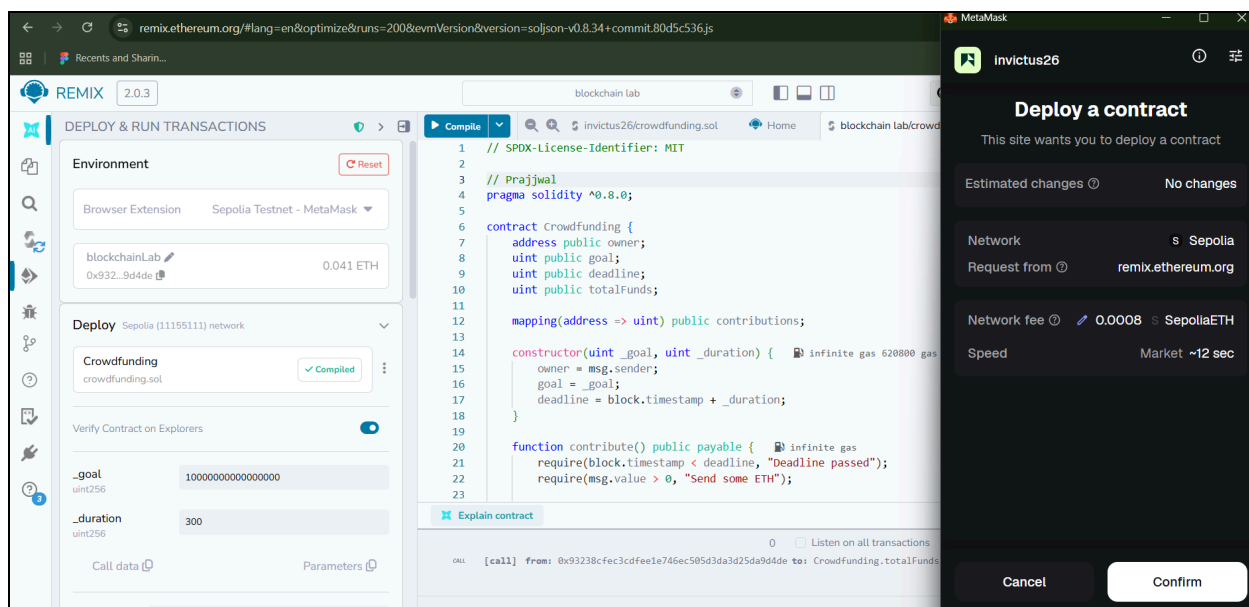
function getBalance() public view returns (uint) {
    return address(this).balance;
}
}

```

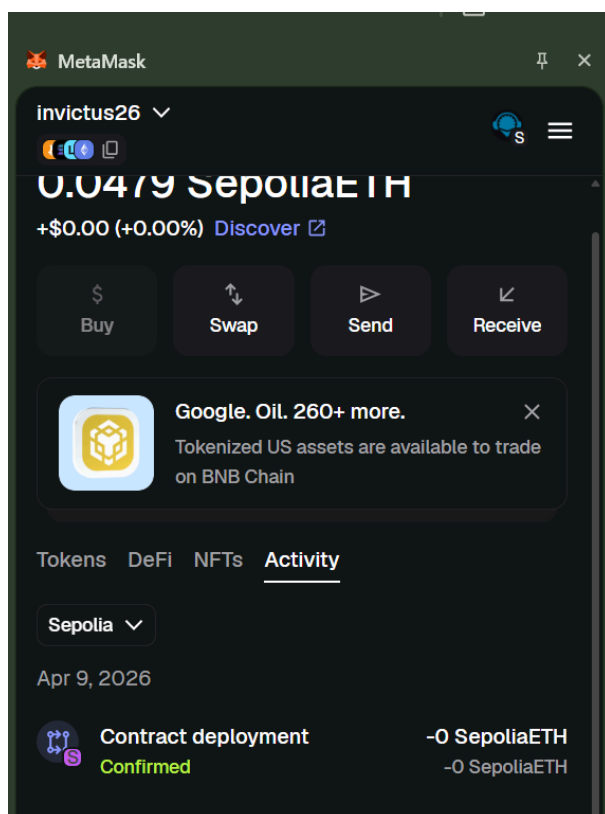
Step 3: save it as **crowdfunding.sol** and compile it



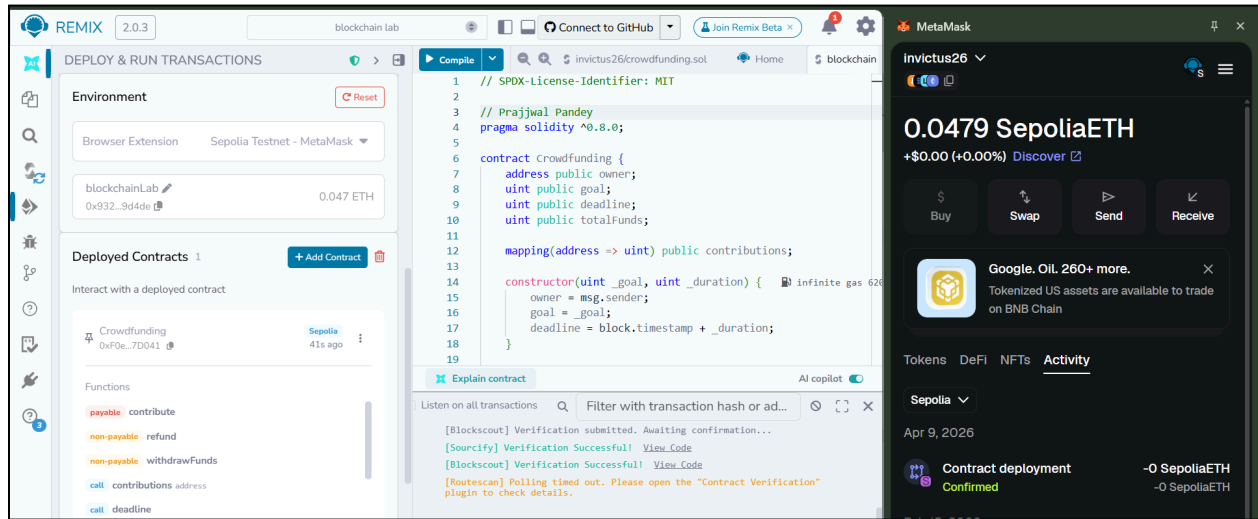
Step 4: deploy the contract with an initial goal and time limit. Confirm the transaction from metamask



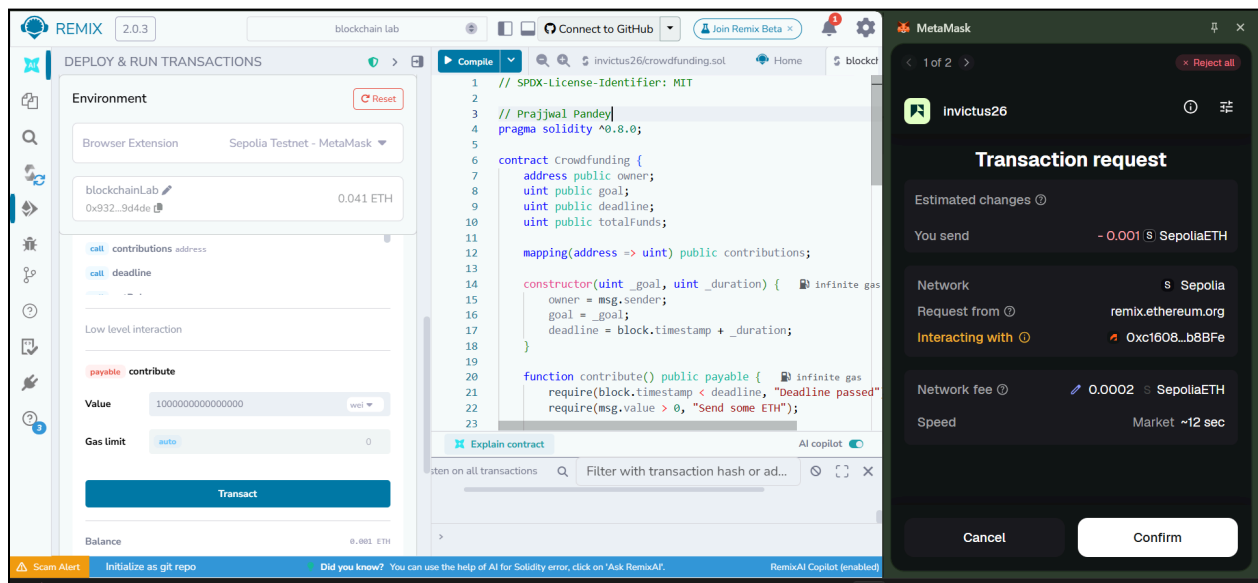
After deployment you can see: transaction recorded in metamask

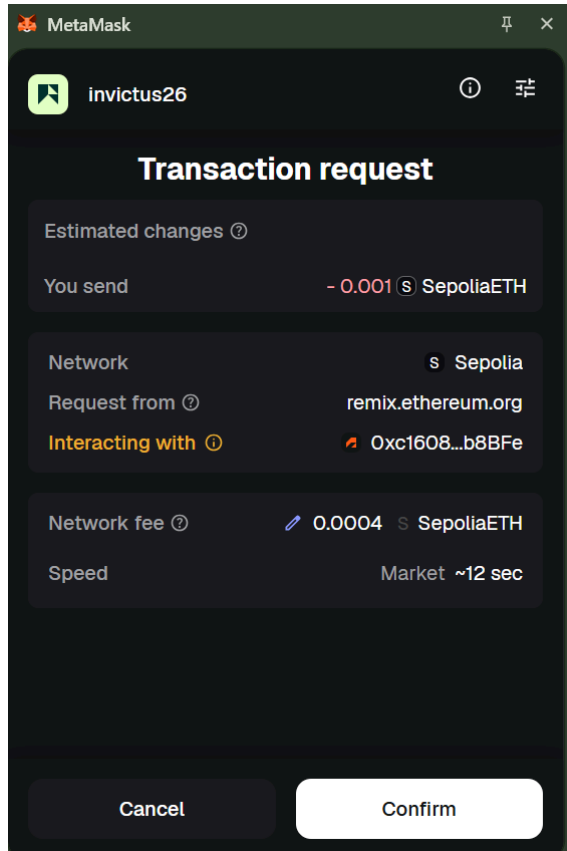


Step 5: After deployment in remix ide, we can see the deployed contract and its functions

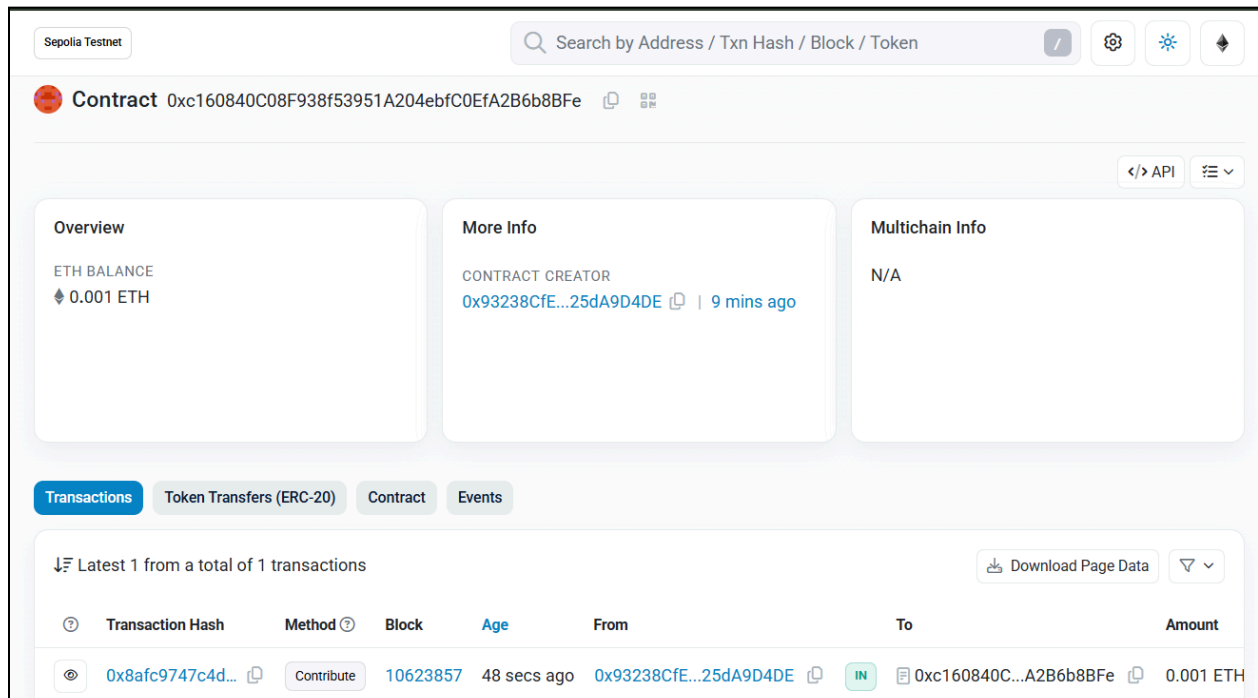


Step 6: Add a contribution of 0.01 ETH = 10000000000000000 wei. Transaction request appears in metamask. Click on confirm

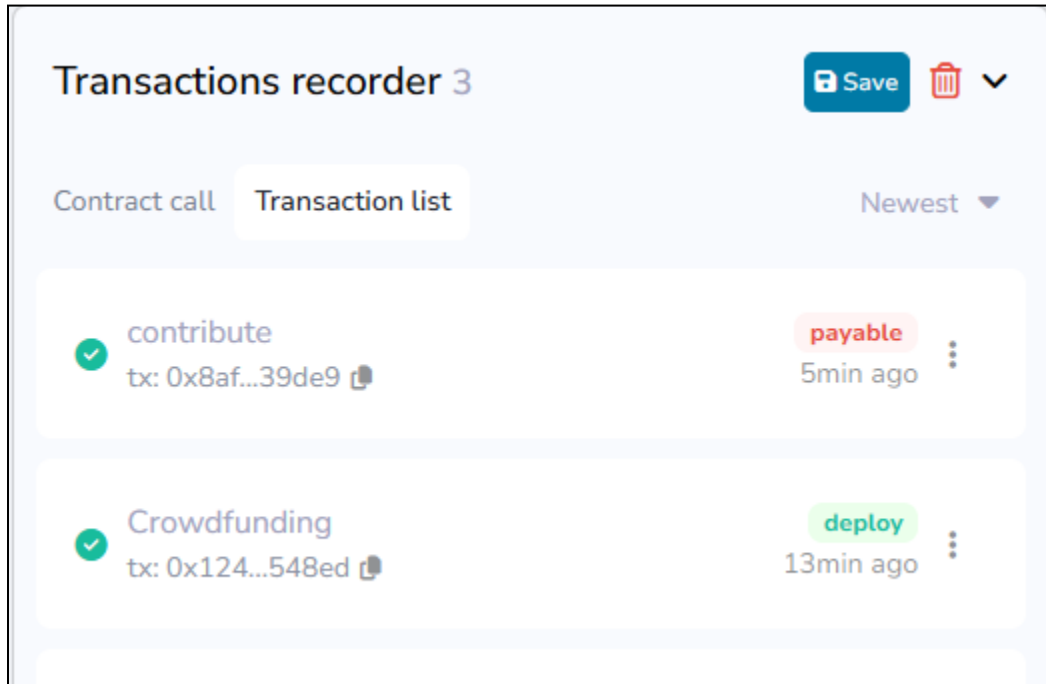




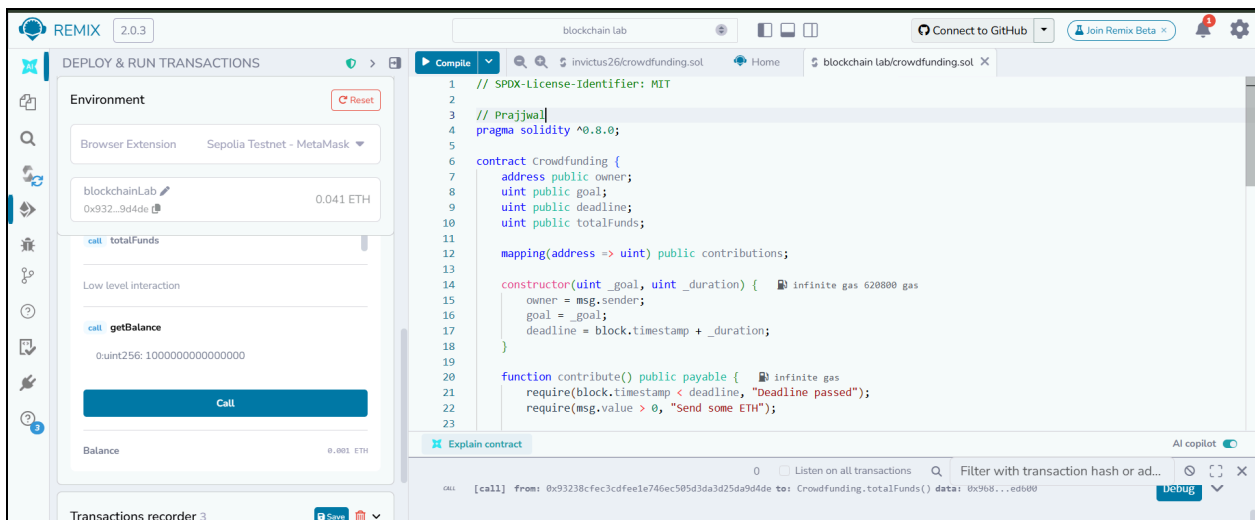
Step 7: copy paste the contracts address in <https://sepolia.etherscan.io/> to see all transactions:



Also can see the transactions under all transactions in remix IDE



Step 8: use getBalance function to check the total funds received



OUTPUT:

call

getBalance

0:uint256: 10000000000000000

Call

Balance

0.001 ETH

Step 9: Paste wallet address from metamask into contributions to see all transactions by the particular use

call

contributions

address

0x93238CfEc3cdfEE1E746eC505d3DA3D25dA9D4DE

0:uint256: 10000000000000000

Call

Step 10: use totalFunds function to see totalFunds recieved

call

totalFunds

0:uint256: 10000000000000000

Call

CONCLUSION:

The integration of MetaMask with Remix IDE offers developers a hands-on approach to understanding how blockchain transactions work in practice. By deploying smart contracts on a testnet like Sepolia, developers can observe the complete transaction flow — from wallet authorization and gas estimation to block confirmation — without any financial risk. This setup bridges the gap between theoretical knowledge and real-world blockchain development, building familiarity with wallet management, network configuration, and contract interaction in a safe and controlled environment.